In practice: unittest

In practice: `unittest`

- Python includes `unittest` in the standard library.

In practice: `unittest`

- Python includes `unittest` in the standard library.
- You can use it to write tests that check expected behavior and catch regressions.

In practice: `unittest`

- Python includes `unittest` in the standard library.
- You can use it to write tests that check expected behavior and catch regressions.
- Practical workflow:

In practice: `unittest`

- Python includes `unittest` in the standard library.
- You can use it to write tests that check expected behavior and catch regressions.
- Practical workflow:
  - write one or more test cases

In practice: `unittest`

- Python includes `unittest` in the standard library.
- You can use it to write tests that check expected behavior and catch regressions.
- Practical workflow:
  - write one or more test cases
  - run tests from the command line

In practice: `unittest`

- Python includes `unittest` in the standard library.
- You can use it to write tests that check expected behavior and catch regressions.
- Practical workflow:
  - write one or more test cases
  - run tests from the command line
  - test exceptions

In practice: `unittest`

- Python includes `unittest` in the standard library.
- You can use it to write tests that check expected behavior and catch regressions.
- Practical workflow:
  - write one or more test cases
  - run tests from the command line
  - test exceptions
  - if needed, use `setUp` and `tearDown` to create and clean up resources

Defining a `TestCase` subclass

```
import unittest
def add_fish_to_aquarium(fish_list):
    if len(fish_list) > 10:
        raise ValueError("At most 10 fish can be added to the aquarium")
    if "shark" in fish_list:
        return {"tank_a": ["shark"]} # omnomnomnom
    return {"tank_a": fish_list}
class TestAddFishToAquarium(unittest.TestCase):
    def test_add_fish_to_aquarium_success(self):
        actual = add_fish_to_aquarium(fish_list=["shark", "tuna"])
        expected = {"tank_a": ["shark"]}
        self.assertEqual(actual, expected)
```

Executing a `TestCase`

From the same directory as `test_add_fish_to_aquarium.py`:

Executing a `TestCase`

From the same directory as `test_add_fish_to_aquarium.py`:

```
python -m unittest test_add_fish_to_aquarium.py
```

Interpreting passing output

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

Interpreting passing output

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

- `.` means one passing test.

Interpreting passing output

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

- `.` means one passing test.
- `OK` means all discovered tests passed.

Test discovery naming rule

- `unittest` discovers methods whose names begin with `test`.

Test discovery naming rule

- `unittest` discovers methods whose names begin with `test`.
- Example discovered: `test_add_fish_to_aquarium_success`.

Test discovery naming rule

- `unittest` discovers methods whose names begin with `test`.
- Example discovered: `test_add_fish_to_aquarium_success`.
- Example not discovered: `example_test`.

Demo: discovery check

1. Rename `test_add_fish_to_aquarium_success` to `add_fish_to_aquarium_success`.

Demo: discovery check

1. Rename `test_add_fish_to_aquarium_success` to `add_fish_to_aquarium_success`.
2. Run:

   ```
   python -m unittest test_add_fish_to_aquarium.py
   ```

   Record what changed in the output and why.

Demo: bugs in the aquarium

```
def add_fish_to_aquarium(fish_list):
    if len(fish_list) > 10:
        raise ValueError("At most 10 fish can be added to the aquarium")
    if "sharks" in fish_list: # Deliberate bug
        return {"tank_a": ["shark"]} # omnomnomnom
    return {"tank_a": fish_list}

class TestAddFishToAquarium(unittest.TestCase):
    def test_add_fish_to_aquarium_success(self):
        actual = add_fish_to_aquarium(fish_list=["shark", "tuna"])
        expected = {"tank_a": ["shark"]}
        self.assertEqual(actual, expected)
```

Interpreting failing output

```
F
======================================================================
FAIL: test_add_fish_to_aquarium_success (test_add_fish_to_aquarium.TestAddFis
----------------------------------------------------------------------
Traceback (most recent call last):
  File "test_add_fish_to_aquarium.py", line 13, in test_add_fish_to_aquarium_
    self.assertEqual(actual, expected)
AssertionError: {'tank_a': ['shark', 'tuna']} != {'tank_a': ['shark']}
- {'tank_a': ['shark', 'tuna']}
+ {'tank_a': ['shark']}
```

Interpreting failing output

```
F
======================================================================
FAIL: test_add_fish_to_aquarium_success (test_add_fish_to_aquarium.TestAddFis
----------------------------------------------------------------------
Traceback (most recent call last):
  File "test_add_fish_to_aquarium.py", line 13, in test_add_fish_to_aquarium_
    self.assertEqual(actual, expected)
AssertionError: {'tank_a': ['shark', 'tuna']} != {'tank_a': ['shark']}
- {'tank_a': ['shark', 'tuna']}
+ {'tank_a': ['shark']}
```

- `F` means a test failed.

Interpreting failing output

```
F
======================================================================
FAIL: test_add_fish_to_aquarium_success (test_add_fish_to_aquarium.TestAddFis
----------------------------------------------------------------------
Traceback (most recent call last):
  File "test_add_fish_to_aquarium.py", line 13, in test_add_fish_to_aquarium_
    self.assertEqual(actual, expected)
AssertionError: {'tank_a': ['shark', 'tuna']} != {'tank_a': ['shark']}
- {'tank_a': ['shark', 'tuna']}
+ {'tank_a': ['shark']}
```

- `F` means a test failed.
- The traceback points to the assertion and shows an object-level diff.

Testing a function that raises an exception

```
class TestAddFishToAquarium(unittest.TestCase):
    def test_add_fish_to_aquarium_success(self):
        actual = add_fish_to_aquarium(fish_list=["shark", "tuna"])
        expected = {"tank_a": ["shark"]}
        self.assertEqual(actual, expected)
    def test_add_fish_to_aquarium_exception(self):
        too_many_fish = ["shark"] * 25
        with self.assertRaises(ValueError) as exception_context:
            add_fish_to_aquarium(fish_list=too_many_fish)
        self.assertEqual(
            str(exception_context.exception),
            "At most 10 fish can be added to the aquarium"
        )
```

Why `assertRaises` here

- Ensures the right error type is raised (`ValueError`).

Why `assertRaises` here

- Ensures the right error type is raised (`ValueError`).
- Lets us inspect the exception message.

Why `assertRaises` here

- Ensures the right error type is raised (`ValueError`).
- Lets us inspect the exception message.
- Test would fail if:

Why `assertRaises` here

- Ensures the right error type is raised (`ValueError`).
- Lets us inspect the exception message.
- Test would fail if:
  - no exception is raised

Why `assertRaises` here

- Ensures the right error type is raised (`ValueError`).
- Lets us inspect the exception message.
- Test would fail if:
  - no exception is raised
  - a different exception type is raised

Why `assertRaises` here

- Ensures the right error type is raised (`ValueError`).
- Lets us inspect the exception message.
- Test would fail if:
  - no exception is raised
  - a different exception type is raised
  - the message does not match

Assertion methods quick reference

Method	Assertion
`assertEqual(a, b)`	`a == b`
`assertNotEqual(a, b)`	`a != b`
`assertTrue(a)`	`bool(a) is True`
`assertFalse(a)`	`bool(a) is False`
`assertIsNone(a)`	`a is None`
`assertIsNotNone(a)`	`a is not None`
`assertIn(a, b)`	`a in b`
`assertNotIn(a, b)`	`a not in b`

Using `setUp` to create resources

```
import unittest

class FishTank:
    def __init__(self):
        self.has_water = False

    def fill_with_water(self):
        self.has_water = True
```

Using `setUp` to create resources

```
class TestFishTank(unittest.TestCase):
    def setUp(self):
        self.fish_tank = FishTank()

    def test_fish_tank_empty_by_default(self):
        self.assertFalse(self.fish_tank.has_water)

    def test_fish_tank_can_be_filled(self):
        self.fish_tank.fill_with_water()
        self.assertTrue(self.fish_tank.has_water)
```

`setUp` behavior

- `setUp` runs before each test method.

`setUp` behavior

- `setUp` runs before each test method.
- Each test gets a fresh `FishTank`.

`setUp` behavior

- `setUp` runs before each test method.
- Each test gets a fresh `FishTank`.
- This avoids repeated setup code in each test.

Using `tearDown` to clean up resources

```
import os
import unittest

class AdvancedFishTank:
    def __init__(self):
        self.fish_tank_file_name = "fish_tank.txt"
        default_contents = "tuna"
        with open(self.fish_tank_file_name, "w") as f:
            f.write(default_contents)

    def empty_tank(self):
        os.remove(self.fish_tank_file_name)
```

Using **tearDown** to clean up resources

```
class TestAdvancedFishTank(unittest.TestCase):
    def setUp(self):
        self.fish_tank = AdvancedFishTank()

    def tearDown(self):
        self.fish_tank.empty_tank()

    def test_fish_tank_writes_file(self):
        with open(self.fish_tank.fish_tank_file_name) as f:
            contents = f.read()
        self.assertEqual(contents, "tuna")
```

`tearDown` behavior

- `tearDown` runs after each test method.

`tearDown` behavior

- `tearDown` runs after each test method.
- Use it to clean up shared resources (files, DB rows, temp directories).

`tearDown` behavior

- `tearDown` runs after each test method.
- Use it to clean up shared resources (files, DB rows, temp directories).
- This keeps tests independent and repeatable.

Running multiple files

```
python -m unittest discover
```

Running multiple files

```
python -m unittest discover
```

- `discover` finds and runs tests across multiple files.

Running multiple files

```
python -m unittest discover
```

- `discover` finds and runs tests across multiple files.
- Use this once your project has several test modules.

Exercise: run-fail-fix cycle

1. Run the aquarium test file and capture passing output.

Exercise: run-fail-fix cycle

1. Run the aquarium test file and capture passing output.
2. Intentionally change expected output to fail.

Exercise: run-fail-fix cycle

1. Run the aquarium test file and capture passing output.
2. Intentionally change expected output to fail.
3. Run again and read the traceback.

Exercise: run-fail-fix cycle

1. Run the aquarium test file and capture passing output.
2. Intentionally change expected output to fail.
3. Run again and read the traceback.
4. Fix the expectation and return to green.

Exercise: exception test (in class)

- Write `test_add_fish_to_aquarium_exception` yourself.

Exercise: exception test (in class)

- Write `test_add_fish_to_aquarium_exception` yourself.
- Use `assertRaises(ValueError)`.

Exercise: exception test (in class)

- Write `test_add_fish_to_aquarium_exception` yourself.
- Use `assertRaises(ValueError)`.
- Assert the exception message exactly.

Exercise: fixtures (in class or lab)

- Implement a `setUp` test fixture for `FishTank`.

Exercise: fixtures (in class or lab)

- Implement a `setUp` test fixture for `FishTank`.
- Implement `tearDown` cleanup for a file-based class.

Exercise: fixtures (in class or lab)

- Implement a `setUp` test fixture for `FishTank`.
- Implement `tearDown` cleanup for a file-based class.
- Run tests twice to confirm no leftover state.